Module 10: Pointers

10.1 Memory and Addresses

- When a variable is declared, the compiler allocates a specific block of memory to store its value
- Every memory location has a unique numerical address (typically displayed in hexadecimal)
- The address-of operator (&) returns the memory address of a variable

```
int x = 42;
cout << &x; // prints the address, for example 0x61ff08
```

- Different variables will have different addresses

10.2 Pointer Variables

- A pointer is a variable whose value is a memory address
- It "points to" another variable by storing that variable's address
- Declaration

```
int *ptr; // ptr is a pointer to an integer
double *dp; // dp is a pointer to a double
```

- Initializing a pointer with the address of a variable

```
int x = 42;
int *ptr = &x; // ptr holds the address of x
```

- The null pointer: a pointer that does not point to anything
- In modern C++: int *ptr = nullptr;
- In older C++: int *ptr = NULL;
- Dereferencing a null pointer causes a crash; always check before dereferencing
- Uninitialized pointers contain a random garbage address; using them causes undefined behavior

10.3 Dereferencing Pointers

- The dereference operator (*) accesses the value stored at the address a pointer holds
- Reading the value

```
int x = 42;
int *ptr = &x;
cout << *ptr; // prints 42
```

- Modifying the value through the pointer

```
*ptr = 100;
cout << x; // now x is 100
```

- The dereference operator * has two different meanings depending on context
- In a declaration: int *ptr; means ptr is a pointer variable

- In an expression: *ptr; means access the value at the address ptr holds

10.4 Pointer Arithmetic

- Adding an integer to a pointer moves it forward by that many elements (not bytes)
- The pointer advances by sizeof(the type it points to) bytes
- int *ptr; ptr + 1 moves the pointer 4 bytes forward (size of int)
- double *ptr; ptr + 1 moves 8 bytes forward
- This is how arrays can be traversed with pointers

10.5 Pointers and Arrays

- The name of an array is actually a constant pointer to its first element
- int arr[5]; - arr is equivalent to &arr[0]
- Accessing elements using pointer arithmetic

```
cout << *(arr + 2); // same as arr[2]
```

- Arrays and pointers can be used interchangeably in many situations

10.6 Dynamic Memory Allocation

- Memory allocated at compile time is called static memory (stack)
- Memory allocated during program execution is called dynamic memory (heap)
- Allocating a single variable dynamically

```
int *ptr = new int;      // allocate one integer
*ptr = 50;               // store a value
delete ptr;             // free the memory when done
ptr = nullptr;          // good practice: reset after delete
```

- Allocating an array dynamically

```
int *arr = new int[100]; // allocate array of 100 integers at runtime
delete[] arr;            // use delete[] for arrays, not delete
```

- Memory leaks: forgetting to delete dynamically allocated memory causes the program to waste memory over time
- Always pair every new with a delete and every new[] with a delete[]